

Dental Booking Platform

Development Phases Roadmap — Team Reference

Product: Multi-tenant dental booking SaaS (AWS)

Date: May 2026

Audience: Engineering, product, stakeholders

Estimated timeline: 10–14 weeks to full platform with billing

Repository (planned): dental-booking-platform

1. Product model (agreed)

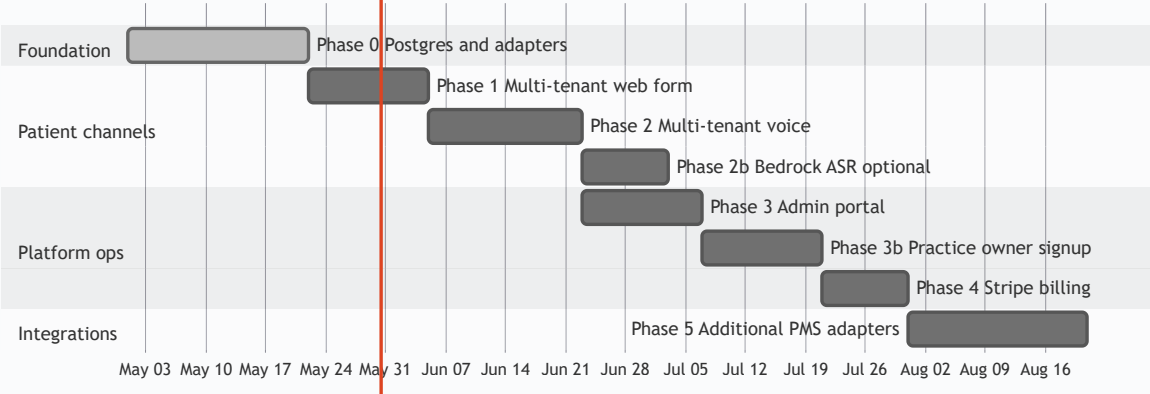
Topic	Decision
Who registers	Practice owners only — not patients
Users per practice	One admin user at signup; bots handle patient-facing tasks
Multi-tenant	Each clinic = one tenant ; many clinics on one platform
Voice (default)	Amazon Connect + Lex bots (booking, admin, up to 8 per tenant)
Web booking form	Optional add-on — <code>features.webForm: false</code> at signup; enabled on request
PMS integration	HTTP APIs via <code>PmsAdapter</code> — Oryx first; no copy of Oryx internal DB schema
GoHighLevel	Out of scope — not in AWS product roadmap
Cloud	AWS only — ECS, Connect, Lex, RDS, S3, Secrets Manager, Bedrock (optional)

Oryx PMS schema reference (e.g. industry model with patients/appointments tables) is useful for understanding dental data — *not* required to run our platform. We store **tenant config** only; booking uses Oryx MyChart APIs.

2. Phase overview

Phase	Duration	Status	One-line outcome
0 Foundation	2–3 weeks	Implemented	Postgres, tenants, Oryx adapter, internal APIs
1 Web form	~2 weeks	Next	<code>/book/[slug]</code> , branding, webForm gate
2 Voice multi-tenant	2–3 weeks	Planned	DID → tenant+bot; Lex uses platform config
2b Bedrock ASR	1–2 weeks	Optional	Name/email correction via LLM
3 Admin portal	~2 weeks	Planned	Operator tools: tenants, bots, phones, assets
3b Practice signup	2–3 weeks	Planned	Self-service registration + admin login
4 Stripe billing	1–2 weeks	Planned	Subscriptions, feature sync, suspend
5 More PMS	Ongoing	Planned	Open Dental, Dentrix, custom API, RPA

Dental Booking Platform Roadmap



3. Phase details

Phase 0 — Foundation

Implemented

2–3 weeks · Prerequisite for all phases

Goal: Multi-tenant data model and shared booking engine; Smile Squad as tenant #1.

Deliverables:

- Postgres + Prisma (local Docker; RDS via Terraform in follow-up PR)
- Tables: `tenants`, `tenant_pms_config`, `tenant_bots`, `tenant_phone_numbers`, `subscriptions` (stub)
- `TenantService`, `PmsAdapter` + `OryxAdapter`, per-tenant operatory rules JSON
- APIs: `GET /api/internal/resolve-phone`, `/api/t/[slug]/public-config|availability|book`
- Legacy routes preserved: `/api/availability`, `/api/web/book`, `POST /api/book` (secured with `CONNECT_WEBHOOK_SECRET`)
- Seed tenant `smilesquad` (Oryx realm `smilesquadpd`)
- Lex handler unchanged except Phase 2 TODO — still calls platform HTTP APIs

Not in Phase 0: Slug web UI, Lex tenant lookup, Stripe, self-service signup, admin portal, Bedrock.

Phase 1 — Multi-tenant web booking form

Next

~2 weeks

Goal: Public web booking per clinic slug; web form only when add-on enabled.

Deliverables:

- `/book/[slug]` — dynamic branding from `tenants.branding`
- Redirect `/book` → `/book/smilesquad` (backward compat)
- Wire UI to `/api/t/[slug]/availability` and `/api/t/[slug]/book`
- Feature gate: **403** if `features.webForm` is false
- Per-tenant access code (`web_form_access_code`)
- Default new tenants: `webForm: false` until ops/Stripe enables add-on

Who uses it: Patients (no login) book into tenant's Oryx realm via web.

Phase 2 — Multi-tenant voice bots

Planned

2–3 weeks

Goal: One platform serves many clinics on voice; DID selects tenant + bot.

Deliverables:

- Lex Lambda calls `resolve-phone` at call start (DID → tenant, bot, operatory rules, prompts)
- Replace hardcoded “Smile Squad” copy with tenant/bot `prompts_config`
- Remove duplicated operatory logic from Lambda — use platform rules
- Support up to **8 bots per tenant** (booking, admin, FAQ, etc.)
- Connect: map each DID to `tenant_phone_numbers`
- `POST /api/book` with `X-Tenant-Slug` where needed

Who uses it: Patients call practice number; bots book via same Oryx APIs as web.

Phase 2b — Bedrock ASR correction

Optional

1–2 weeks (after Phase 2)

Goal: Improve voice capture of names, email, and correction turns.

- `POST /api/internal/normalize-slot` (platform secret)
- Amazon Bedrock for structured extraction; mandatory read-back before book
- Per-tenant feature flag; heavier use on admin/FAQ bot
- Booking APIs remain deterministic (no LLM in final book call)

Phase 3 — Admin portal (operator)

Planned

~2 weeks

Goal: Internal/ops team manages tenants without SQL.

- `/admin` — tenant CRUD, PMS config, operatory rules editor
- Bot registry (create/disable, Lex IDs, Lambda ARNs, prompts JSON)
- Phone → bot mapping
- Enable/disable `features.voice` and `features.webForm` (web form add-on)
- S3 upload for logos / hero images
- Booking health dashboard (success/fail per tenant)

Onboarding v1: Operator creates tenant (manual) — no public signup yet.

Phase 3b — Practice owner self-service Planned

2–3 weeks (after Phase 3 or with Phase 4)

Goal: Practice owner registers, picks PMS (Oryx first), gets one admin login.

- New table: `platform_users` (email, password, role, `tenant_id`)
- Registration flow: practice info → select PMS type → provision tenant
- Default: `voice: true`, `webForm: false`
- “Request web form” workflow → ops approves → flip feature + access code
- Practice admin dashboard (limited): view status, request add-ons, basic config

Not included: Patient accounts, staff RBAC beyond one admin (later).

Phase 4 — Stripe subscriptions Planned

1–2 weeks

- Stripe products: voice plans, web form add-on
- `POST /api/webhooks/stripe` — update `subscriptions`, sync `tenants.features`
- Suspend tenant on `past_due` / `canceled` (403 on book + voice routing)
- Optional: self-serve checkout linked to Phase 3b signup

Phase 5 — Additional PMS adapters Ongoing

Ongoing after Phase 4

- `OpenDentalAdapter`, `DentrixAdapter`, `CustomApiAdapter`
- Same `PmsAdapter` interface — UI and Lex unchanged
- RPA/SQS workers for PMS without public APIs (future; not FastAPI rewrite in current plan)

4. What we store vs what Oryx stores

Our Postgres (platform)	Oryx / PMS (via API)
Tenant slug, branding, features, bots, DIDs	Patients, charts, appointments
<code>realm</code> , operatory rules, adapter config	Providers, operatories, schedule slots
Stripe subscription status	Clinical, billing, insurance (not mirrored)
Platform admin users (Phase 3b)	Practice staff in PMS (<code>staff_users</code> in PMS model only)

5. Channels end-state

Channel	Entry	Phase
Web booking	<code>/book/{slug}</code>	Phase 1 (APIs ready in Phase 0)
Voice booking	Connect DID → Lex → <code>/api/book</code>	Phase 2
Practice admin	<code>/admin</code> or signup portal	Phase 3 / 3b
Internal routing	<code>/api/internal/resolve-phone</code>	Phase 0 ✓

6. Infrastructure (by phase)

Component	Phase
Docker Postgres (local)	0 ✓
RDS Postgres + Terraform	Follow-up PR after 0
ECS Fargate + ALB (existing)	Ongoing
S3 tenant assets	3
ACM + Route53 TLS	3+
Bedrock	2b (optional)

7. Success criteria (release-ready platform)

- Second clinic onboarded with **DB config only** (no app redeploy)

- `/book/{slug}` shows correct branding; books correct Oryx realm
- Call to Clinic B's DID uses B's bot + PMS config
- Voice-only tenant gets **403** on web form
- Web form add-on enables `webForm` + access code without redeploy
- Practice owner can register (Phase 3b) and manage one admin account
- Stripe cancellation suspends tenant within one webhook cycle (Phase 4)
- Second PMS adapter plugs in without UI/Lex flow changes (Phase 5)

8. Explicitly out of scope

- GoHighLevel integration and `tenant_ghl_locations`
- Patient registration on our platform
- Replicating Oryx's 22-table PMS database in our Postgres
- FastAPI rewrite (stay on Next.js full-stack)
- DynamoDB, EventBridge, SQS RPA workers (until Phase 5+ path)

Phase 0 code location: `oryx-agent/` today; planned clean repo `dental-booking-platform` (no GHL) for team delivery.